

1 DesignGym Research Note

1.1 Conclusion

DesignGym should optimize for a **repeatable practice loop**, not for course breadth. The smallest meaningful learner submission is a structured written design that makes requirements, classes, responsibilities, interactions, edge cases, and trade-offs visible. Feedback should combine deterministic checks with judgment-oriented review, preserve the learner's evidence, and end with a small next-attempt focus.

For CipherSchools, this is a natural companion to industry-relevant software-development education. A learner may understand a design pattern in a lesson and still struggle to apply it to an ambiguous problem. DesignGym provides the missing practice-and-feedback layer without attempting to replace CipherSchools' broader technical, aptitude, and soft-skills programs.

1.2 What existing practice approaches emphasize

Low-level design interview resources consistently frame the task as an exercise in code organization. The learner is expected to identify objects, define interfaces and relationships, and explain how the design can absorb change. Hello Interview's LLD guide also emphasizes clarifying scope before designing, responsibility assignment, maintainability, and communication rather than pattern-name memorization.¹ A second useful pattern comes from broader system-design practice. ByteByteGo describes design interviews as collaborative problem solving with no single perfect answer. Its framework starts with requirements and assumptions, moves to a shared blueprint, and uses feedback and concrete use cases to decide what deserves a deeper dive.² Although the scale differs from LLD, the learning implication is the same: a learner improves faster when the product captures reasoning, not only a final diagram or a reference answer.

The review workflow offers a third signal. Google's engineering guidance treats review as a mechanism for continuous improvement rather than a search for perfection. It recommends weighing comments by importance and grounding design disagreements in engineering principles rather than personal preference.³ Claude Code Review shows a more operational version of the same idea: findings are verified, deduplicated, ranked, and posted against the specific lines where issues were found, with a summary and severity information.⁴

1.3 Product implications

These approaches point to four product decisions.

First, **requirements and assumptions must be first-class fields**. A blank canvas encourages the learner to jump directly to class names. A short problem brief and explicit assumptions make the reasoning boundary reviewable.

Second, **structured written design is the right MVP format**. A diagram is useful for relationships, but it is weak evidence for responsibilities, edge-case behavior, and trade-offs unless the learner also explains them. A written format is easier to persist, validate, compare across attempts, and extend to diagram or code submissions later.

Third, **deterministic and judgment-based feedback should be separate**. Required sections, minimum evidence, duplicate names, and basic structural completeness can be checked reliably. Responsibility

quality, coupling, and trade-offs require a rubric evaluator. Presenting both under one unexplained score would blur confidence and make the product feel arbitrary.

Fourth, **the reference answer should not be the only definition of correctness**. LLD problems have multiple defensible designs. The product should show evidence, concern, and suggestion for each criterion. It should also preserve prior attempts so a learner can see whether the same weakness recurs and whether the next attempt addresses it.

1.4 Deliberate scope and gap

DesignGym is intentionally not an LMS, code runner, diagram editor, or production coaching marketplace. Its gap is narrower: many practice experiences stop at a prompt, a model solution, or a generic score. This prototype instead makes the improvement loop visible: choose a problem, write a structured design, submit, inspect evidence-linked feedback, see recurring weak criteria, and retry without losing history. The evaluator is deterministic in this prototype so that the same submission produces a repeatable demonstration. The interface is replaceable, so a future LLM or human evaluator can be added without rewriting the attempt lifecycle or the learner workspace.

1.5 Implementation alignment

The final implementation preserves these research decisions as explicit code boundaries. `Attempt` is a guarded monolithic state machine, `Submission` supports a future class-diagram strategy, and `EvaluatorStrategy` supports deterministic, human, rule-based, or LLM implementations. `EvaluationEngine` handles the asynchronous boundary and falls back to deterministic feedback when a primary evaluator fails. Rubric output is parsed as structured JSON with the fields `Criterion`, `Score`, `Evidence`, `Concern`, and `Suggestion`, so the review remains explainable rather than becoming an opaque model response.

This architecture is intentionally sized for the assignment: it demonstrates change handling and failure behavior without introducing microservices, queues, or an operationally heavy evaluation platform. SQL-backed attempts and authenticated learner accounts extend the prototype beyond the minimum brief while keeping the core learner loop understandable.

1.6 References